8. grep, ripgrep mindset, and find

08. grep, ripgrep mindset, and find

Phase: Phase 3 — Text, pipes, and discovery
Estimated time: 1–2 days
Slug: `08-grep-find`

Learning objectives

- Search file contents with grep
- Find files by name/type/time with find
- Combine find -exec / xargs safely
- Avoid unsafe parsing of filenames with spaces

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Almost every troubleshooting session boils down to two questions: “which lines contain this?” and “which files match that?”. `grep` searches inside content, `find` searches the tree by name, type, size, time and permissions. Together they let you find an error in gigabytes of logs or a stray file on a full disk in seconds.

Core concepts

`grep PATTERN [FILE...]` prints matching lines. The pattern is a *basic regular expression* by default; `-E` enables extended regex (`+`, `?`, `|`, `()`), `-F` treats it as a fixed string, and `-P` (GNU) enables Perl-style regex.

grep option	Meaning
`-i`	Ignore case
`-n`	Show line numbers
`-v`	Invert: lines that do not match
`-r` / `-R`	Recurse into directories
`-l`	Only list file names with a match
`-c`	Count matching lines
`-w`	Match whole words
`-o`	Print only the matched part
`-q`	Quiet, exit status only
`-A/-B/-C N`	Show N lines after/before/around

grep exit codes: 0 match found, 1 no match, 2 error.

`find PATH TESTS ACTION` walks a tree evaluating tests left to right.

```
find /var/log -type f -name "*.log" -mtime -1 -size +1M -print
       |         |          |             |        |       |
     start    regular     name glob   modified   bigger   action
     point     files                  < 1 day    than 1MB (default)

find ... -exec cmd {} \;     run cmd once per file
find ... -exec cmd {} +      batch many files per cmd
find ... -print0 | xargs -0  safe with spaces in names
```

Useful tests: `-type f|d|l`, `-name` (case-sensitive glob; quote it), `-iname`, `-mtime N`, `-mmin N`, `-size +10M`, `-user`, `-perm`, `-maxdepth N`, `-empty`. Combine with `-a`, `-o`, `!` and parentheses (escaped).

A common pattern is matching TODO markers in source: lines containing the word `TODO` (often with a colon), typically `grep -rn "TODO" src/`.

Worked examples

Search text:

```
printf 'alpha\nTODO: fix\nbeta\ntodo later\n' > notes.txt
grep -n TODO notes.txt
# sample output: 2:TODO: fix
grep -in todo notes.txt
# sample output: 2:TODO: fix
# sample output: 4:todo later
grep -vc TODO notes.txt
# sample output: 3
```

Recursive search over a project:

```
grep -rn --include='*.py' 'TODO' .
# sample output: ./app/main.py:12:    # TODO: handle errors
grep -rl 'TODO' . | wc -l
# sample output: 4
```

Use grep in a condition:

```
if grep -q '^root:' /etc/passwd; then echo has-root; fi
# sample output: has-root
```

Find files:

```
find /var/log -maxdepth 1 -type f -name '*.log' -size +100k
# sample output: /var/log/dpkg.log
find . -type f -mtime -1 -name '*.txt'
# sample output: ./notes.txt
find . -type f -empty -delete
```

Run commands on results safely:

```
find . -name '*.tmp' -print0 | xargs -0 rm -f
find . -type f -name '*.sh' -exec chmod +x {} +
```

Extended regex:

```
grep -E '^(ERROR|WARN)\b' app.log | head
# sample output: ERROR disk full
# sample output: WARN retry 3
```

Mini-lab

1. `mkdir -p /tmp/lab8/src && cd /tmp/lab8`.
2. Create sample files: `printf 'x\n# TODO: a\n' > src/a.py; printf 'y\n' > src/b.py; printf 'TODO fix\n' > src/c.txt`.
3. `grep -rn TODO src` and read the file:line:text format.
4. `grep -rl TODO src` and `grep -rL TODO src` to list matching and non-matching files.
5. `grep -rn --include='*.py' TODO src` to limit by extension.
6. `find src -type f -name '*.py'` then `find src -type f -newer src/a.py`.
7. `find /etc -maxdepth 1 -name 'host*' 2>/dev/null`.
8. `find src -type f -exec grep -H TODO {} +` and compare with step 3.

Decision guide / failure modes

Use `grep` when you know the content; use `find` when you know metadata (name, age, size, owner). Use `grep -r` for quick searches, `find ... -exec grep` or `xargs` when you need fine control over which files. For large code bases `rg` (ripgrep) is faster, but `grep` exists everywhere.

Common failures:

- Unquoted globs: `find . -name *.txt` is expanded by the shell if matching files exist in the current directory; write `'*.txt'`.
- Forgetting `-type f` and getting directories too.
- `xargs` without `-0` breaking on names with spaces.
- Treating grep's exit status 1 as an error in `set -e` scripts; use `|| true` when "no match" is acceptable.
- Regex confusion: in basic grep `+` and `|` are literal unless escaped; use `-E`.
- `grep -r pattern` in a directory with huge binary or generated files; use `--exclude-dir`.

Common mistakes

- Parsing `ls` output in scripts
- `find | xargs` without `-print0`/`-0`
- Recursive grep on huge binary trees without limits

Exercises

Work in `exercises/08-grep-find/`. Read that folder's `README.md` fully before coding.

```
cd exercises/08-grep-find
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement match_todo(line). It takes one line of text and returns True if the line contains the marker TODO as a whole word, uppercase exactly, and False otherwise. "TODO: fix" and "# TODO fix this" match; "todo" in lower case does not, and neither do words that merely contain the letters such as "TODOS" or "AUTODOC". Word boundaries mean the characters on each side of TODO are not letters, digits or underscores (start/end of line counts as a boundary). This mimics grep -w TODO and is a good use of a small regular expression.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/08-grep-find/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Why `-print0`?
2. Find files modified in last day?
3. grep flag to show line numbers?

Further reading

- https://man7.org/linux/man-pages/man1/find.1.html